

Native AI Integration for Model-Based Systems Engineering: Three Layers that Make It Work

****Author:**** Colin Dempsey

****Publisher:**** Software Engineering Institute (SEI), Carnegie Mellon University – SEI Blog

****Published:**** September 2, 2026

****Source URL:**** <https://www.sei.cmu.edu/blog/native-ai-integration-for-model-based-systems-engineering-three-layers-that-make-it-work/>

****Archived for citation purposes (thesis: master-thesis, bibkey sei-native-ai) on 2026-09-19. Full text below is the readable-extracted content of the live page; formatting (tables, code blocks) simplified from the original HTML.****

The appeal of applying artificial intelligence (AI) to model-based systems engineering (MBSE) is easy to understand. Formal architecture definitions, traceable requirements, behavior models, analysis, and verification evidence take time to develop and maintain. AI could plausibly reduce some of that effort, but plausibility is not evidence.

For program managers and engineering leaders, the opportunity is broader than faster model authoring. Integrating AI directly into model development can reduce the effort required to create, update, and reconcile engineering artifacts. It can shorten the time between an engineering change and feedback from the modeling toolchain, surface inconsistencies before integration and test, and preserve traceable evidence for technical decisions. The management value should therefore be judged by whether teams make better-informed decisions sooner and reduce rework and technical risk, while engineers retain responsibility for review, analysis, and assurance.

The harder question is how to integrate AI without weakening engineering rigor. Four questions guide our work:

- How can engineering teams natively integrate AI into MBSE modeling?
- Where does that integration add measurable value?
- What does an AI-augmented MBSE workflow look like?
- What does that workflow look like in practice?

As detailed in this post, we investigated these questions by building a three-layer integration architecture, evaluating it in a controlled SysML v2 model-generation benchmark, and examining one large benchmark task involving a four-drone aerial survey swarm.

SysML v2 in Brief

Systems Modeling Language version 2, commonly referred to as SysML v2, is the Object Management Group's formal language for describing systems through requirements, structure, behavior, analysis, verification, and stakeholder views. It provides both graphical and textual notation over the same underlying model. This work focuses on the textual notation because model files can be reviewed as text, stored in version control, compared in pull requests, and processed by automated tools.

Two ideas make the short code sample below readable. A **definition**, or ``def``, declares a reusable type, while a **usage** places that type in a particular model context. A package supplies a namespace. Typed attributes can use quantities and units from the standard libraries.

...

```
package DroneSwarmExample {  
    private import ScalarValues::*;
```

```

enum def FlightMode {
    enum idle;
    enum takeoff;
    enum survey;
    enum returnToHome;
}

part def FlightController {
    attribute droneId : Integer;
    attribute activeMode : FlightMode;
}

part def Drone {
    part controller : FlightController;
}

part drone : Drone;
}..

```

Here, `FlightMode`, `FlightController`, and `Drone` are definitions. The nested controller is a usage that composes the drone from a flight controller, while drone is a top-level usage of the complete type. The identifier and operating mode are typed values rather than unqualified properties.

Textual notation makes SysML v2 accessible to coding assistants, but it does not make the language informal. Generated model text must still conform to the grammar, resolve its references, and satisfy the language's semantic rules. That requirement is why a language-aware modeling service and a callable validation interface are central to the architecture.

The Architecture

Native integration means placing the AI assistant inside the engineering workspace rather than treating it as a separate chat surface. The assistant works in the same repository as the model, retrieves project guidance, edits model artifacts, invokes the same validation command used by engineers and continuous integration, and responds to diagnostics. The engineer remains responsible for intent, review, and engineering decisions.

The workspace combines three separable layers: an AI coding assistant, a formal modeling toolchain, and a curated knowledge base with workflow skills.

- **AI coding assistant.** The assistant receives direction in natural language and operates on version-controlled artifacts. The architecture is not tied to a particular assistant client or AI model. Our current implementation uses Visual Studio Code as the primary engineering environment. Through editor extensions and integrated terminals, Claude Code, OpenAI Codex, and Continue each operate on the same repository and connect to the same modeling tools and project knowledge.

- **Language-aware modeling toolchain.** The general requirement is a SysML v2 language server, or an equivalent language-aware service, that can parse SysML v2 model files, resolve references, apply grammar and semantic rules, and return diagnostics. For agentic and automated use, those capabilities should be exposed through a noninteractive interface, ideally a command-line interface. This lets assistants, engineers, and continuous integration (CI) jobs invoke the same check.

Our implementation uses Sensmetry's Syside for both interactive editor support and automated model checking. In Visual Studio Code, Syside communicates through

h the Language Server Protocol (LSP), a standard interface that supplies features such as diagnostics, reference resolution, and navigation. The AI assistants do not call that interface directly; instead, they invoke model checks through the `syside check` command. A clean run establishes conformance to the checks performed by Syside. It does not establish that the model is complete, reflects the correct engineering decisions, or has been verified.

- **Knowledge and workflow layer.** The SEI assembled a knowledge base as a version-controlled collection of Markdown files containing SysML v2 language guidance, validation-rule explanations, modeling patterns, and examples. A repository-local Model Context Protocol (MCP) server catalogs these files under stable topic names and exposes focused search and retrieval tools to AI assistants. In this implementation, the knowledge service is not a large language model (LLM) or a knowledge graph, it is an interface to SEI-curated reference material. Separate version-controlled skill files define how assistants use the retrieved guidance, modify model artifacts, invoke validation and analysis tools, and respond to their results.

MCP is a good fit for this work because it separates the curated knowledge source from any particular AI assistant or model. Compatible clients can call the same server and retrieval-tool contract without requiring the SEI to build and maintain a separate integration for each assistant. This design aligns with an emerging cross-vendor pattern: Microsoft, AWS, Google, and Atlassian all provide MCP-based mechanisms through which AI clients can search or retrieve managed knowledge.

The implementation is intentionally flexible rather than tied to a fixed product stack. It requires three core capabilities: an assistant that can work on repository artifacts, language-aware modeling tools with callable validation and analysis interfaces, and project knowledge that can be retrieved during the task. The same pattern can extend to other modeling languages and toolchains; the SEI is currently applying it to the Architecture Analysis and Design Language (AADL) and its supporting analysis tools.

(Figure 1 in the original post: "AI-augmented MBSE workspace in Visual Studio Code." The three columns map to the assistant, modeling-toolchain, and knowledge layers described above. SEI Nexus OPAL is an SEI-managed service for hosted models. Earlier experiments also integrated Claude Code and Anthropic models through the same interfaces shown for OpenAI Codex.)

What the Validation Loop Demonstrated

To distinguish the effects of language-tool feedback from those of the knowledge base and workflow skills, the SEI evaluated the prototype workspace across three experimental arms using the same SysML v2 modeling tasks. All three arms used Anthropic's Claude Opus 4.6.

- **Baseline** uses a single generation pass with no tools.
- **Command-line interface (CLI) only** allows generation followed by a `syside check` feedback loop, without the SEI knowledge base or skills.
- **Full tooling** adds the MCP knowledge service, modeling skills, and the same Syside validation loop.

The experiment covered eight tasks at two scales: 40 small-task outputs per arm and 24 large-task outputs per arm. The baseline serves as a single-pass reference point. The two iterative conditions test what changes occur when the assistant is able to respond to language-tool diagnostics, with or without project-specific guidance.

Table 1: Small-task results (n=40 per arm)

Arm	Errors / 100 Lines	Lines	Iterations	Wall time	Pattern measure, m
-----	--------------------	-------	------------	-----------	--------------------

	mean (SD), max 100	Tokens (est.)
Baseline	3.7 309 n/a 87s 78.3 (7.7) 21K	
CLI-only	0 277 2.1 178s 71.7 (6.3) 33K	
Full tooling	0 302 1.7 181s **94.1 (5.0)** 71K	

****Table 2: Large-task results (n=24 per arm)****

Arm	Errors / 100 Lines	Lines	Iterations	Wall time	Pattern measure, mean (SD), max 100	Tokens (est.)
Baseline	25.6 3303 n/a 599s 78.3 (7.7) 59K					
CLI-only	0 2426 3.4 914s 71.7 (6.3) 106K					
Full tooling	0 2458 2.6 **772s** **94.1 (5.0)** 143K					

The project-defined pattern measure, implemented as a Node.js script, combines 29 measures into a 0-100 composite:

- 12 structural measures covering model breadth, construct richness, and connectivity
- 13 measures covering traceability, canonical modeling patterns, and model organization
- 4 task-specific acceptance measures

Both iterative conditions ended with zero reported Syside errors and warnings at both scales. In these runs, language-tool feedback allowed the assistant to identify and remove parser and semantic diagnostics. Validation alone did not materially improve the project-defined pattern measure. Compared with CLI-only, full tooling was modestly higher on small tasks and substantially higher on large tasks, where the mean rose from 71.7 to 94.1. Curated knowledge and workflow skills supplied guidance for model structure, traceability, and task coverage that the validator did not require.

The result separates two useful effects: language tooling checks conformance, while curated knowledge and workflows influence the broader modeling patterns captured by this project-defined measure. A score of 100 would mean that every benchmark measure awarded full credit, but it would not prove engineering correctness, completeness, or fitness for use. The experiment used one model family, one language, one validator, and a controlled task corpus; independent reproduction remains necessary.

Grounding the AI: Knowledge Base and Workflow Skills

An LLM can produce SysML that looks plausible while omitting model relationships, project conventions, or evidence needed by reviewers. Language tooling catches malformed syntax and some semantic violations, but it cannot decide whether the model reflects the intended system or whether a program's preferred patterns have been applied consistently.

The knowledge layer addresses that gap by making relevant guidance retrievable in the local workspace. The current repository includes SysML v2 and KerML rule descriptions, OMG training lessons and examples, project-authored pattern guides, shared libraries, and a reusable project template. The MCP server exposes this material through stable topics so the assistant can retrieve a specific reference instead of relying on general model memory. Because both the service and its sources are version controlled, changes can be reviewed and tied to the results they influence.

For example, if Syside reports that a `satisfy` relationship violates type conformance, the assistant can retrieve the traceability guidance, determine whether `satisfy` or `allocate` is appropriate at that location, update the model, and rerun `syside check`. In SysML v2, `satisfy` declares that a design element fulfi

lls a requirement, while `allocate` maps one model element to another, such as a assigning a behavior to the component responsible for performing it. Syside remains the authority for its diagnostics; the knowledge base helps the LLM interpret those diagnostics and supplies project patterns that are outside the parser's responsibility.

That distinction matters because many modeling decisions are valid syntax but weak engineering. A parser cannot choose an appropriate decomposition level, decide whether a relationship should express satisfaction or allocation, distinguish a source fact from an inference, or determine which stakeholder view will support a review. Curated guidance gives the assistant explicit criteria for those choices, while source citations and repository history leave the resulting decisions open to inspection.

Curated knowledge alone, however, does not define a repeatable way to apply it. The knowledge base provides the **what**: rules, examples, patterns, and decision criteria. Workflow skills provide the **how**: which context to retrieve for a task, what sequence to follow, when to invoke deterministic tools, what evidence to retain, and when to stop. Together they turn guidance into a repeatable retrieve-edit-check workflow.

The SEI developed the workflow skills using a standard, file-based skill framework and authoring guidance available from the AI coding tools used in this work. Each skill encodes an engineering workflow the team wanted to experiment with. Table 3 summarizes the 10 skills and their roles, spanning knowledge priming, model development, diagnostic repair, review, analysis, reverse engineering, view generation, code generation, and document generation. The skills are version-controlled orchestration instructions, not replacements for deterministic tools: Syside performs parsing and validation, test runners execute tests, and renderers produce outputs.

****Table 3: Current SEI-developed SysML v2 workflow skills****

Skill	Role in the workflow
knowledge-primer-sysml	Loads the relevant language, tooling, and modeling context before work begins.
model-sysml	Builds models in dependency order and runs validation after changes.
fix-sysml	Interprets diagnostics, retrieves matching guidance, applies fixes, and revalidates.
review-sysml	Combines validator statistics with a structured, evidence-based model review.
analyze-sysml	Guides constraints, analysis cases, trade studies, and traceability-gap checks.
explain-sysml	Retrieves references and examples to explain language and model constructs.
reverse-engineer-sysml	Maps existing source code into a model with source-location metadata.
views-sysml	Creates stakeholder-specific diagrams, tables, and matrices.
codegen-sysml	Maps validated model constructs into a prototype implementation.
docgen-sysml	Builds model-derived documentation while leaving narrative claims for review.

The table is a division of responsibility rather than a claim that every task uses every skill. A model-development task may begin with the primer and modeling workflow, call the fixing workflow only when diagnostics appear, and use the views workflow when stakeholder presentations are required. Each transition is visible in the work record.

Worked Example: Modeling a Four-Drone Survey Swarm

To show the workflow on a larger task, the SEI examined one benchmark assignment for a four-drone aerial survey swarm and ground control station. The prompt description below called for coordinated survey-grid assignment, multispectral imaging, real-time kinetic (RTK) positioning-assisted navigation, mesh communication, separation assurance, battery and weather-triggered return-to-home, lost-link recovery, and waypoint-triggered cargo delivery. The requested deliverable spanned eight coordinated SysML v2 files.

Prompt for Four Drone Survey Swarm

Model a 4-drone aerial survey swarm with ground control station in SysML v2. Each drone carries a multi-spectral camera, GPS/INS with RTK correction ($\pm 2\text{cm}$ accuracy), an anemometer, a mesh radio for inter-drone communication, a cargo drop mechanism with confirmation sensor, and a battery with state-of-charge monitoring. The ground control station (GCS) assigns survey grid segments to each drone, displays real-time fleet telemetry, and manages mission lifecycle. A deconfliction engine (running on GCS with cooperative inputs from each drone) maintains $\geq 50\text{m}$ horizontal and $\geq 20\text{m}$ vertical separation, dynamically replanning paths when projected conflicts are detected within a 30-second lookahead window. Return-to-home triggers: battery below 20% reserve, operator command, or sustained wind $> 40\text{km/h}$ for > 5 seconds (measured by onboard anemometer). Lost-link protocol: if a drone loses GCS contact for > 30 seconds, it autonomously climbs to a deconfliction altitude and returns to its last-known home waypoint. The mesh network allows drones to relay telemetry for others that are out of direct GCS range. Cargo drop is triggered at a GPS waypoint with release confirmation from a mechanical sensor.

Produce 8 `.sysml` files following the template structure: `types.sysml` (enums, metadata), `requirements.sysml` (30+ requirements), `structure.sysml` (all parts/ports/connections), `behavior.sysml` (state machines + action flows), `analysis.sysml` (battery endurance + coverage + separation budget), `verification.sysml` (test cases), `integration.sysml` (4-drone fleet instantiation), `views.sysml` (viewpoints). Aim for 3000-4000 lines total across all files.

In the full-tooling condition, the assistant developed types, requirements, structure, behavior, analysis, verification, integration, and views in dependency order. The resulting artifact contains 3,186 lines, or 2,135 non-blank, non-comment lines. It includes 35 requirement definitions, 22 part definitions, 16 action definitions, five state definitions containing 34 states, seven analysis definitions, 16 verification definitions, 38 satisfaction relationships, six viewpoints, and 16 view definitions. After two validation iterations, `syside check --warnings-as-errors` completed with zero reported errors or warnings. The artifact received 97.2 of 100 on the project-defined pattern measure.

The model is easier to inspect when one concern is followed across its system engineering concern. The excerpts below use deconfliction and minimum separation as that thread.

****Requirements.**** The requirements model gives the separation obligation a formal identifier, readable text, priority, and safety metadata:

...

```
requirement def <'DCN-001'> MinimumSeparation {
  doc /* The deconfliction subsystem shall maintain a minimum horizontal
        separation of 30 meters between any two drones in the swarm
        during all phases of flight. */

  @Priority { level = PriorityLevel::critical; }
  @SafetyCriticality {
    level = SafetyLevel::safetyCritical;
    standardRef = "MIL-STD-882E";
    hazard = "Mid-air collision between swarm members";
```

```
}
}...
```

****Structure.**** The structure model comprises four typed drone usages with a ground control station (GCS) and makes command, telemetry, mesh, and coordination paths explicit:

```
...
part def DroneSwarmSystem {
  part drones : Drone [4];
  part gcs : GCS;

  connection droneToGcsCommand
    connect gcs.commandOut to drones.commandIn;
  connection droneToGcsTelemetry
    connect drones.telemetryOut to gcs.telemetryIn;
  connection meshNetwork
    connect drones.meshPort to gcs.meshPort;
  connection interDroneCoordination
    connect drones.coordinationPort to drones.coordinationPort;
}...
```

****Behavior.**** The behavior model decomposes conflict resolution into strategy selection, maneuver computation, clearance checking, and execution:

```
...
action def ResolveConflict {
  in conflictingDroneId : Integer;
  in strategy : DeconflictionStrategy;
  out resolutionManeuver : String;

  action selectStrategy;
  action computeManeuver;
  action validateClearance;
  action executeManeuver;

  first selectStrategy then computeManeuver;
  first computeManeuver then validateClearance;
  first validateClearance then executeManeuver;
}...
```

****Analysis.**** The analysis model defines a swarm-level separation case and supplies inputs for pairwise distance and time-to-conflict calculations:

```
...
analysis def SeparationAnalysis {
  subject swarm : DroneSwarmSystem;

  objective separationObjective {
    doc /* Verify minimum 30m separation is maintained in
        converging trajectory scenarios. */
  }

  calc pairwiseDistance : SeparationDistance {
    in drone1X = 0.0;
    in drone1Y = 0.0;
    in drone1Z = 50.0;
    in drone2X = 25.0;
    in drone2Y = 15.0;
```

```

        in drone2Z = 50.0;
        return distanceM;
    }

    calc timeToViolation : TimeToConflict {
        in relativePositionM = 35.0;
        in relativeVelocityMs = 5.0;
        in minimumSeparationM = 30.0;
        return timeToConflictSec;
    }

    return worstCaseSeparationM : Real;
}...

```

****Verification.**** The verification model links the separation requirement usage to an ordered test procedure:

```

...
verification def SeparationMaintenanceTest {
    subject testSwarm : DroneSwarmSystem;

    objective {
        verify separationReq;
    }

    action configureSwarm {
        out swarmReady : Boolean;
    }
    action executeConvergingPaths {
        out pathsExecuted : Boolean;
    }
    action measureSeparation {
        out minSeparationM : Real;
    }
    action evaluateResult {
        out testPassed : Boolean;
    }

    first configureSwarm then executeConvergingPaths;
    first executeConvergingPaths then measureSeparation;
    first measureSeparation then evaluateResult;
}...

```

Together, the excerpts show how AI can produce a navigable first-pass model in which a requirement, system topology, behavior, analysis case, and verification procedure can be reviewed together. The model also demonstrates why the validation loop is necessary but not sufficient. The prompt requested 50-meter horizontal and 20-meter vertical separation, while the model specifies 30 and 10 meters. The prompt also requested lost-link return to home (RTH) after more than 30 seconds, while the model specifies detection in three seconds followed by a 60-second loiter. The generated structure also omits the requested multispectral camera, RTK receiver, anemometer, weather-abort logic, and survey-grid assignment, and it places deconfliction computers onboard the drones rather than locating the engine at the ground station.

Review of the generated model identified issues beyond source fidelity. Although analysis and verification elements are present, they remain planning structures rather than completed evidence. Calculation definitions declare inputs and returns but contain no executable formulas. The analysis return is not bound to a calculation result, and no requirement or analysis constraints are evaluated. Phys

ical quantities use `Real` with units embedded in names, instead of using SysML v2's quantity-and-unit libraries. The verification case defines an objective and sequenced steps but no formal verdict expression or test result.

A clean validation run therefore shows only that the model passes the tool's syntax and semantic checks. Source reconciliation, executable analysis, verification results, and engineering review are still required to determine whether the model is correct and fit for use. These safeguards will remain necessary even as LLM capabilities advance. Continued refinement of the knowledge base and workflow guidance, informed by findings such as these, should measurably improve the quality and completeness of models produced through AI-augmented development activities.

Closing Insights on AI-Augmented MBSE

The four opening questions lead to the following four conclusions:

- First, engineering teams can natively integrate AI into MBSE by placing the assistant inside the version-controlled modeling workspace, where it can retrieve project guidance, edit artifacts, invoke the modeling toolchain, and respond to the same diagnostics used by engineers and continuous integration (CI) tools.
- Second, the controlled benchmark shows a measurable but bounded value. In these runs, iterative language server feedback removed reported parser and semantic diagnostics, while the full toolchain produced stronger scores on the modeling patterns encoded in the project's rubric. Validation and guidance improved different aspects of the result.
- Third, an AI-augmented MBSE workflow is more than an assistant and a prompt. It combines a callable language-aware service, curated knowledge, repeatable workflow skills, deterministic tools, and human review.
- Fourth, the four-drone swarm task shows that combination in practice: a detailed system description becomes a coordinated model of requirements, structure, behavior, analysis and verification plans, integration, traceability, and stakeholder views. It also shows how cross-model review can expose source mismatches and incomplete evidence that language validation alone cannot detect. The approach does not transfer engineering accountability to the LLM. A clean model still requires expert review, analysis, and verification.

We invite teams to share your own experience with AI-augmented MBSE and explore collaboration opportunities at the SEI to evaluate the impact of AI on your own MBSE workflows. For more information or to collaborate with the SEI, please send an email to info@sei.cmu.edu.